

Лабораторная работа №14

Динамическое программирование. Задача о рюкзаке. Наибольшая общая подпоследовательность (LCS)

Цель работы:

1. Изучить основные принципы динамического программирования: оптимальная подструктура, перекрывающиеся подзадачи, мемоизация
2. Реализовать классическую задачу о рюкзаке (0/1) с использованием табличного метода ДП
3. Реализовать алгоритм поиска наибольшей общей подпоследовательности (LCS) для двух строк
4. Научиться восстанавливать решение (какие предметы выбраны, какая подпоследовательность)
5. Сравнить рекурсивный подход с мемоизацией и итеративный подход

Теоретическое обоснование

1. Динамическое программирование

Динамическое программирование (ДП) - это метод решения задач, который разбивает сложную задачу на более простые подзадачи, решает каждую подзадачу один раз и сохраняет результат для последующего использования.

Основные принципы ДП:

Принцип	Описание
Оптимальная подструктура	Оптимальное решение задачи содержит оптимальные решения подзадач

Принцип	Описание
Перекрывающиеся подзадачи	Одна и та же подзадача возникает многократно
Мемоизация	Сохранение результатов подзадач для повторного использования

Способы реализации ДП:

Способ	Описание	Сложность	Память
Рекурсия + мемоизация	Сверху вниз (top-down)	Зависит от задачи	$O(\text{таблица})$
Табличный метод	Снизу вверх (bottom-up)	Зависит от задачи	$O(\text{таблица})$

2. Задача о рюкзаке (0/1 Knapsack)

Постановка задачи: Дано N предметов, каждый предмет имеет вес $\text{weight}[i]$ и стоимость $\text{value}[i]$. Дан рюкзак вместимостью W . Нужно выбрать предметы так, чтобы их суммарный вес не превышал W , а суммарная стоимость была максимальной. Каждый предмет можно взять только один раз (0/1).

Визуализация:

Предметы:

0: вес=2, ценность=3

1: вес=3, ценность=4

2: вес=4, ценность=5

3: вес=5, ценность=8

Оптимальный набор:

взять предметы 0 и 3:

суммарный вес = 2+5=7

суммарная ценность = 3+8=11

Таблица ДП (W=10, N=4):

i\w	0	1	2	3	4	5	6	7	8	9	10
0	0	0	3	3	3	3	3	3	3	3	3
1	0	0	3	4	4	7	7	7	7	7	7
2	0	0	3	4	5	7	8	9	9	9	9
3	0	0	3	4	5	8	8	11	12	13	13

Рекурсивное решение (без оптимизации):

```
int knapsackRecursive(int i, int W) {  
    if (i < 0 || W == 0) return 0;  
  
    if (weight[i] > W) {  
        return knapsackRecursive(i - 1, W);  
    } else {  
        return max(knapsackRecursive(i - 1, W),  
                   knapsackRecursive(i - 1, W - weight[i]) + value[i]);  
    }  
}
```

Рекурсивное решение с мемоизацией (top-down):

vector<vector<int>> memo,

```
int knapsackMemo(int i, int W) {  
    if (i < 0 || W == 0) return 0;  
    if (memo[i][W] != -1) return memo[i][W];  
  
    if (weight[i] > W) {  
        memo[i][W] = knapsackMemo(i - 1, W);  
    } else {  
        memo[i][W] = max(knapsackMemo(i - 1, W),  
                          knapsackMemo(i - 1, W - weight[i]) + value[i]);  
    }  
}
```

```

        knapsackMemo(i - 1, W - weight[i] + value[i]);
    }
    return memo[i][W];
}

```

Табличное решение (bottom-up):

```

int knapsackTable(int N, int W) {
    vector<vector<int>> dp(N + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= N; i++) {
        for (int w = 1; w <= W; w++) {
            if (weight[i-1] > w) {
                dp[i][w] = dp[i-1][w];
            } else {
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w - weight[i-1]] + value[i-1]);
            }
        }
    }

    return dp[N][W];
}

```

Оптимизация по памяти (одномерный массив):

```

int knapsackOptimized(int N, int W) {
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < N; i++) {
        for (int w = W; w >= weight[i]; w--) {
            dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
}

```

```

return dp[W];
}

Восстановление выбранных предметов:
vector<int> getSelectedItems(int N, int W, const vector<vector<int>>& dp) {
    vector<int> selected;
    int w = W;

    for (int i = N; i > 0; i--) {
        if (dp[i][w] != dp[i-1][w]) {
            selected.push_back(i-1); // предмет i-1 выбран
            w -= weight[i-1];
        }
    }

    return selected;
}

```

3. Наибольшая общая подпоследовательность (LCS)

Постановка задачи: Даны две строки (последовательности). Найти их наибольшую общую подпоследовательность (не обязательно непрерывную).

Визуализация LCS:

Строка X: "ABCBDAB"

Строка Y: "BDCABA"

LCS: "BCBA" или "BCAB"

Символы совпадают:

B → B, C → C, B → B

Длина LCS = 4

Таблица LCS (7×7):

	0	1	2	3	4	5	6	7	(Y)
0	0	0	0	0	0	0	0	0	
1	0	0	0	0	1	1	1	1	
2	0	1	1	1	1	2	2	2	
3	0	1	2	2	2	2	3	3	
4	0	1	2	2	2	3	3	4	
5	0	1	2	3	3	3	4	4	
6	0	1	2	3	4	4	4	4	
7	0	1	2	3	4	4	5	5	

Рекурсивное решение:

```

int lcsRecursive(const string& X, const string& Y, int i, int j) {
    if (i == 0 || j == 0) return 0;

    if (X[i-1] == Y[j-1]) {
        return 1 + lcsRecursive(X, Y, i-1, j-1);
    } else {
        return max(lcsRecursive(X, Y, i-1, j),
                    lcsRecursive(X, Y, i, j-1));
    }
}

```

Табличное решение (bottom-up):

```

int lcsTable(const string& X, const string& Y) {
    int m = X.length();
    int n = Y.length();

    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }

    return dp[m][n];
}

```

Восстановление LCS:

```
string getLCS(const string& X, const string& Y, const vector<vector<int>>&
dp) {
    int i = X.length();
    int j = Y.length();
    string lcs = "";

    while (i > 0 && j > 0) {
        if (X[i-1] == Y[j-1]) {
            lcs = X[i-1] + lcs;
            i--; j--;
        } else if (dp[i-1][j] > dp[i][j-1]) {
            i--;
        } else {
            j--;
        }
    }

    return lcs;
}
```

Примеры выполнения практической части

Пример 1. Задача о рюкзаке (базовый уровень)

Задача: Реализовать решение задачи о рюкзаке табличным методом.

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
```

```

int knapsack(int W, const vector<int>& weights, const vector<int>& values)
{
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (weights[i-1] > w) {
                dp[i][w] = dp[i-1][w];
            } else {
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w - weights[i-1]] + values[i-1]);
            }
        }
    }

    return dp[n][W];
}

int main() {
    vector<int> weights = {2, 3, 4, 5};
    vector<int> values = {3, 4, 5, 8};
    int W = 10;

    int maxValue = knapsack(W, weights, values);
    cout << "Максимальная стоимость: " << maxValue << endl;

    return 0;
}

```

Ожидаемый вывод:

Максимальная стоимость: 13

Пример 2. Задача о рюкзаке с восстановлением (средний уровень)

Задача: Реализовать задачу о рюкзаке с восстановлением выбранных предметов.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
using namespace std;
```

```
struct KnapsackResult {
    int maxValue;
    vector<int> selectedItems;
};
```

```
KnapsackResult knapsackWithRecovery(int W, const vector<int>& weights,
const vector<int>& values) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (weights[i-1] > w) {
                dp[i][w] = dp[i-1][w];
            } else {
                dp[i][w] = max(dp[i-1][w],
                    dp[i-1][w - weights[i-1]] + values[i-1]);
            }
        }
    }
}
```

```

// Восстановление выбранных предметов
vector<int> selected;
int w = W;
for (int i = n; i > 0; i--) {
    if (dp[i][w] != dp[i-1][w]) {
        selected.push_back(i-1);
        w -= weights[i-1];
    }
}
reverse(selected.begin(), selected.end());

return {dp[n][W], selected};
}

int main() {
    vector<int> weights = {2, 3, 4, 5};
    vector<int> values = {3, 4, 5, 8};
    int W = 10;

    auto result = knapsackWithRecovery(W, weights, values);

    cout << "Максимальная стоимость: " << result.maxValue << endl;
    cout << "Выбранные предметы (индексы): ";
    for (int idx : result.selectedItems) {
        cout << idx << " ";
    }
    cout << endl;
    cout << "Вес выбранных предметов: ";
    int totalWeight = 0;

```

```

for (int idx : result.selectedItems) {
    totalWeight += weights[idx];
}

cout << totalWeight << " (из " << W << ")" << endl;

return 0;
}

```

Ожидаемый вывод:

Максимальная стоимость: 13

Выбранные предметы (индексы): 0 3

Вес выбранных предметов: 7 (из 10)

Пример 3. Наибольшая общая подпоследовательность (продвинутый уровень)

Задача: Реализовать LCS с восстановлением строки.

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

```

```

using namespace std;

```

```

struct LCSResult {
    int length;
    string subsequence;
};

```

```

LCSResult lcs(const string& X, const string& Y) {
    int m = X.length();
    int n = Y.length();

```

```
vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
```

```
for (int i = 1; i <= m; i++) {  
    for (int j = 1; j <= n; j++) {  
        if (X[i-1] == Y[j-1]) {  
            dp[i][j] = dp[i-1][j-1] + 1;  
        } else {  
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);  
        }  
    }  
}
```

```
// Восстановление подпоследовательности
```

```
string lcsStr = "";  
int i = m, j = n;  
while (i > 0 && j > 0) {  
    if (X[i-1] == Y[j-1]) {  
        lcsStr = X[i-1] + lcsStr;  
        i--; j--;  
    } else if (dp[i-1][j] > dp[i][j-1]) {  
        i--;  
    } else {  
        j--;  
    }  
}
```

```
return {dp[m][n], lcsStr};  
}
```

```
int main() {
```

```

string X = "ABCBDAAB";
string Y = "BDCABA";

auto result = lcs(X, Y);

cout << "Строка X: " << X << endl
cout << "Строка Y: " << Y << endl
cout << "Длина LCS: " << result.length() << endl
cout << "LCS: " << result.subsequence() << endl

return 0;
}

```

Ожидаемый вывод:

Строка X: ABCBDAAB

Строка Y: BDCABA

Длина LCS: 4

LCS: BCBA

Пример 4. Сравнение подходов (продвинутый уровень)

Задача: Сравнить производительность рекурсивного и табличного подходов для LCS.

```

#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <functional>
#include <map>

```

```

using namespace std;
using namespace chrono;

```

// Рекурсивный LCS (экспоненциальный)

```
int lcsRecursive(const string& X, const string& Y, int i, int j) {  
    if (i == 0 || j == 0) return 0;  
    if (X[i-1] == Y[j-1]) {  
        return 1 + lcsRecursive(X, Y, i-1, j-1);  
    }  
    return max(lcsRecursive(X, Y, i-1, j),  
               lcsRecursive(X, Y, i, j-1));  
}
```

// LCS с мемоизацией (top-down)

```
int lcsMemo(const string& X, const string& Y, int i, int j,  
            vector<vector<int>>& memo) {  
    if (i == 0 || j == 0) return 0;  
    if (memo[i][j] != -1) return memo[i][j];  
  
    if (X[i-1] == Y[j-1]) {  
        memo[i][j] = 1 + lcsMemo(X, Y, i-1, j-1, memo);  
    } else {  
        memo[i][j] = max(lcsMemo(X, Y, i-1, j, memo),  
                          lcsMemo(X, Y, i, j-1, memo));  
    }  
    return memo[i][j];  
}
```

// LCS табличный (bottom-up)

```
int lcsTable(const string& X, const string& Y) {  
    int m = X.length();  
    int n = Y.length();  
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
```

```

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (X[i-1] == Y[j-1]) {
            dp[i][j] = dp[i-1][j-1] + 1;
        } else {
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
}

return dp[m][n];
}

```

```

int main() {
    string X = "ABCB DAB";
    string Y = "BDCABA";

    cout << "=== Сравнение подходов LCS ===" << endl;
    cout << "X: " << X << " (длина " << X.length() << ")" << endl;
    cout << "Y: " << Y << " (длина " << Y.length() << ")" << endl;

    // Рекурсивный (только для маленьких строк)
    auto start = high_resolution_clock::now();
    int lenRec = lcsRecursive(X, Y, X.length(), Y.length());
    auto end = high_resolution_clock::now();
    auto timeRec = duration_cast<microseconds>(end - start).count();
    cout << "\nРекурсивный: длина = " << lenRec << ", время = " << timeRec
    << " мкс" << endl;

    // С мемоизацией

```

```

vector<vector<int>> memo(X.length() + 1, vector<int>(Y.length() + 1, -
1));

start = high_resolution_clock::now();
int lenMemo = lcsMemo(X, Y, X.length(), Y.length(), memo);
end = high_resolution_clock::now();
auto timeMemo = duration_cast<microseconds>(end - start).count();
cout << "С мемоизацией: длина = " << lenMemo << ", время = " <<
timeMemo << " мкс" << endl;

```

```

// Табличный
start = high_resolution_clock::now();
int lenTable = lcsTable(X, Y);
end = high_resolution_clock::now();
auto timeTable = duration_cast<microseconds>(end - start).count();
cout << "Табличный: длина = " << lenTable << ", время = " << timeTable
<< " мкс" << endl;

```

```

return 0;

```

```

}

```

Ожидаемый вывод:

=== Сравнение подходов LCS ===

X: ABCBDAB (длина 7)

Y: BDCABA (длина 6)

Рекурсивный: длина = 4, время = 45 мкс

С мемоизацией: длина = 4, время = 15 мкс

Табличный: длина = 4, время = 8 мкс

Индивидуальные задания

Часть 1. Задача о рюкзаке

Каждый вариант содержит задание реализовать решение задачи о рюкзаке с указанными особенностями:

№ вар.	Размерность	Особенности
1	2D таблица	Только максимальная стоимость
2	2D таблица	С восстановлением предметов
3	2D таблица	Веса и ценности - целые числа
4	2D таблица	Веса и ценности - вещественные
5	1D оптимизация	Только максимальная стоимость
6	1D оптимизация	С восстановлением предметов
7	Рекурсия + мемоизация	Только максимальная стоимость
8	Рекурсия + мемоизация	С восстановлением предметов
9	Все три подхода	Сравнение производительности
10	2D таблица	Ограниченный рюкзак (не более К предметов)
11	2D таблица	Рюкзак с несколькими ограничениями (вес + объём)
12	1D оптимизация	Вывод всех оптимальных наборов
13	Рекурсия + мемоизация	Непрерывный рюкзак (дробный)

№ вар.	Размерность	Особенности
14	2D таблица	Зависимые предметы (взять A → можно взять B)
15	2D таблица	Максимизация веса при фиксированной стоимости

Часть 2. Наибольшая общая подпоследовательность (LCS)

Каждый вариант содержит задание реализовать LCS с указанными особенностями:

№ вар.	Особенности	Дополнительное задание
1	Табличный метод	Только длина LCS
2	Табличный метод	Восстановление строки
3	Табличный метод	Все LCS (все варианты)
4	Табличный метод	LCS для трёх строк
5	Рекурсивный (без оптимизации)	Сравнение производительности
6	Рекурсия + мемоизация	Сравнение производительности
7	Табличный метод	LCS для строк с кириллицей
8	Табличный метод	LCS для массивов чисел

№ вар.	Особенности	Дополнительное задание
9	Табличный метод	LCS с весами символов
10	Табличный метод	LCS с ограничением на длину
11	Табличный метод	LCS с пропусками (gap penalty)
12	Табличный метод	Наибольшая общая подстрока (непрерывная)
13	Табличный метод	LCS для векторов строк
14	Табличный метод	LCS для списков (шаблонный)
15	Табличный метод	LCS с восстановлением и визуализацией таблицы

Часть 3. Применение ДП

Задание для всех вариантов:

1. Реализовать программу, которая:
2. Запрашивает у пользователя:
3. Для задачи о рюкзаке: количество предметов, вместимость, вес и ценность каждого предмета
4. Для LCS: две строки
5. Вычисляет оптимальное решение
6. Выводит результат (максимальную стоимость и выбранные предметы / длину LCS и саму подпоследовательность)

7. Выводит таблицу ДП для наглядности

Пример входных данных для рюкзака:

Количество предметов: 4

Вместимость рюкзака: 10

Предмет 0: вес=2, ценность=3

Предмет 1: вес=3, ценность=4

Предмет 2: вес=4, ценность=5

Предмет 3: вес=5, ценность=8

Пример входных данных для LCS:

Строка X: ABCBDAB

Строка Y: BDCABA

Контрольные вопросы

1. Что такое динамическое программирование? Какие принципы лежат в его основе?
2. В чём разница между рекурсивным подходом с мемоизацией (top-down) и табличным (bottom-up)?
3. Как формулируется задача о рюкзаке (0/1)?
4. Какова временная сложность решения задачи о рюкзаке методом ДП?
5. Почему для оптимизации памяти нужно проходить по весам в обратном порядке?
6. Как восстановить выбранные предметы после заполнения таблицы ДП?
7. Как формулируется задача LCS?
8. Какова временная сложность алгоритма LCS?
9. В чём разница между подпоследовательностью и подстрокой?
10. Как восстановить LCS после заполнения таблицы?
11. Какие ещё задачи решаются с помощью ДП?

12. В чём отличие задачи о рюкзаке от задачи о рюкзаке с повторениями?
13. Можно ли решить LCS за меньшее время, чем $O(m \times n)$?
14. Как адаптировать ДП для LCS трёх строк?
15. Как модифицировать задачу о рюкзаке для случая, когда предмет можно взять несколько раз?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (ДП, задача о рюкзаке, LCS)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода)

Анализ сложности алгоритмов

Сравнительная таблица подходов (при наличии)

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Задача о рюкзаке	40
1.1	Реализация алгоритма	15
1.2	Корректность работы	10

№	Критерий	Баллы
1.3	Восстановление предметов (по варианту)	10
1.4	Оптимизация памяти (по варианту)	5
2	Наибольшая общая подпоследовательность (LCS)	40
2.1	Реализация алгоритма	15
2.2	Корректность работы	10
2.3	Восстановление подпоследовательности	10
2.4	Дополнительные возможности (по варианту)	5
3	Применение и анализ	10
3.1	Демонстрационная программа	5
3.2	Анализ сложности	5
4	Оформление и отчёт	10
4.1	Код отформатирован, есть комментарии	5

№	Критерий	Баллы
4.2	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Неправильное заполнение таблицы ДП	-15
Неверное восстановление выбранных предметов/LCS	-15
Неверная сложность алгоритма	-10
Нет демонстрационной программы	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60